

De Lijn Route Suggestions

Feature documentation for De Lijn setup, route suggestions, public-transport matching, route-page UI, map shortcuts, live pickup sharing, and manual testing.

Current version Updated after the latest pull on 2 June 2026. The styling is restored to a visible yellow/black guide format, while preserving the newer routing content.		
Backend De Lijn client, route suggestion service, helper endpoints, and matching integration.	Frontend Route planner page, stop search, skeleton loading, Leaflet map, and map links.	Setup Environment variables, Docker Compose, GitLab variable reminder, and fallback behavior.

This guide explains what is implemented, what depends on a De Lijn API key, what is fallback-based, and what to test during a demo.

Primary files reviewed

- Backend: DeLijnService, TravelGroupRouteSuggestionService, FriendMatchingService, DeLijnApiController, TravelGroupApiController, TravelGroupController.
- Frontend: travelgroup-route-suggestions.js, travelgroup-route-footer.js, travelgroup-map.js, travelgroup-location-share.js, route and travel group templates.
- Configuration and demo: application.properties, docker-compose.yml, Data.sql, localized message files.

1

Overview

This feature lets users view De Lijn bus and tram departure suggestions for a travel group that uses PUBLIC_TRANSPORT. The page combines the travel group's departure point and activity location with live or scheduled De Lijn data, then presents options on a Leaflet map alongside selectable departure cards.

The feature now also connects to public-transport friend matching, compact Google/Apple Maps links, and shared member pickup locations.

Route page

```
/travelgroups/{groupId}/route-suggestions
```

The page shows

- A route search form with origin, destination, and departure time
- A Leaflet map with origin and destination markers
- Loading skeleton cards while data is fetched
- De Lijn departure cards with line, time, stop, and direction
- Google Maps transit links per card
- Fallback messages when De Lijn is not configured or no departures are available

Important scope

The app is not a full door-to-door journey planner by default. Unless DELIJN_API_ROUTE_OPTIONS_PATH is configured with a real route-planner endpoint, it builds best-effort options from De Lijn stops plus scheduled or live departures.

Routing issue status

Issue	Status	Short note
6 - De Lijn API client	Done	API key is environment-based; endpoint paths are configurable.
7 - De Lijn matching	Done	Runs for PUBLIC_TRANSPORT and falls back when data is missing.
8/9 - Suggestions UI	Done	Suggestion cards and join actions remain part of the app.
10 - Tests	Done earlier	Route suggestion extraction also added tests in pulled routing work.
11 - Seed data	Done	Public-transport members, open/full groups, past groups, and Belgian demo locations exist.
12 - Documentation	This PDF	Setup, behavior, limitations, and troubleshooting are combined here.

2

Files Involved

MVC and API Controllers

File	Role
TravelGroupController.java	Loads the group, renders the route suggestion page, and handles shared member locations.
TravelGroupApiController.java	Exposes the JSON route-suggestion endpoint and delegates to the service.
DeLijnApiController.java	Exposes stop suggestions, stop details, departures, probes, and debug helpers.
TravelGroupRouteSuggestionService.java	Main orchestration service for route suggestions.

Services and Configuration

File	Role
DeLijnService.java	Calls De Lijn, parses response shapes, and builds route option DTOs.
DeLijnProperties.java	Configuration properties for base URL, key, timeout, limits, and endpoint paths.
DeLijnConfig.java	Creates the configured RestClient bean.
FriendMatchingService.java	Uses De Lijn evidence for public-transport match scoring.
TravelGroupService.java	Creates/updates group route fields and stores member shared locations.

Frontend

File	Role
travelgroup-route-suggestions.html	Thymeleaf template for the route suggestion page.
travelgroup-route-suggestions.js	Form, map, API calls, cards, stop suggestions, and Google Maps transit links.
travelgroup-route-footer.js	Footer height and horizontal scroll fix for the route page.
travelgroup-map.js	Map picker, reverse geocoding, detail map, and Google/Apple Maps links.
travelgroup-location-share.js	Manual and live member pickup-location sharing.

3

Main User Flow

1. User opens a travel group detail page.
2. If the group uses PUBLIC_TRANSPORT, the route suggestions link is shown.
3. User opens /travelgroups/{groupId}/route-suggestions.
4. TravelGroupController loads the group and renders the page.
5. Thymeleaf writes default route data into data-* attributes on .
6. travelgroup-route-suggestions.js reads those defaults on page load.
7. The script calls the JSON route-suggestion endpoint.
8. TravelGroupApiController delegates to TravelGroupRouteSuggestionService.
9. TravelGroupRouteSuggestionService validates the group, transport mode, coordinates, result count, and De Lijn configuration.
10. DeLijnService searches stops and fetches scheduled or real-time departures.
11. TravelGroupRouteSuggestionService balances bus and tram results.
12. The API returns TravelGroupRouteSuggestionsDto as JSON.
13. The frontend renders the map, summary text, and route cards.

Backend page route

Endpoint

```
GET /travelgroups/{groupId}/route-suggestions
```

The controller loads the group, adds group and routeSuggestionsUrl to the model, then returns travelgroup-route-suggestions.html.

API URL in model

```
/api/travelgroups/{groupId}/route-suggestions?maxResults=8
```

Data stored in the HTML page

Attribute	Source
data-route-suggestions-url	Built by the controller.
data-default-origin-label	group.departureLocation or group.location.
data-default-origin-latitude	group.departureLatitude.
data-default-origin-longitude	group.departureLongitude.
data-default-destination-label	group.activity.location.
data-default-destination-latitude	group.arrivalLatitude or activity.latitude.
data-default-destination-longitude	group.arrivalLongitude or activity.longitude.
data-default-departure-time	group.departureTime.

4

API Route Suggestion Endpoint

Endpoint

```
GET /api/travelgroups/{groupId}/route-suggestions
```

TravelGroupApiController contains no route business logic. It delegates to TravelGroupRouteSuggestionService.getRouteSuggestions(...).

Query parameters

Parameter	Default	Notes
maxResults	4	The route page sends 8.
originLatitude	Group departure latitude	Optional override.
originLongitude	Group departure longitude	Optional override.
originLabel	Group departure location	Optional override.
destinationLatitude	Group arrival or activity latitude	Optional override.
destinationLongitude	Group arrival or activity longitude	Optional override.
destinationLabel	Activity location	Optional override.
departureTime	Group departure time	Optional override.

Validation behavior

Case	Behavior
maxResults out of range	HTTP 400 BAD_REQUEST.
Group not found or not visible	Uses normal travelGroupService.getTravelGroupById behavior.
Group is not PUBLIC_TRANSPORT	Returns supported=false with no options.
Coordinates missing or invalid	Returns supported=true with no options and routeSuggestions.addCoordinates.
De Lijn not configured	Returns configured=false with no options and routeSuggestions.apiKeyMissing.

Main response

Field	Type	Description
supported	boolean	Whether the feature applies to this group.
configured	boolean	Whether De Lijn API settings are present.
message	String	User-facing status or error message.
origin/destination labels and coordinates	String / Double	Route points used by the backend.
transitCoverage	String	Summary text, for example De Lijn covers buses and trams.
options	List	List of DeLijnRouteOptionDto.

5

De Lijn Configuration

Class: DeLijnProperties. Prefix: delijn.api. File: application.properties.

Main properties

```
delijn.api.base-url=${DELIJN_API_BASE_URL:https://api.delijn.be}
delijn.api.api-key=${DELIJN_API_KEY:}
delijn.api.timeout=${DELIJN_API_TIMEOUT:3s}
delijn.api.nearby-stop-radius-meters=${DELIJN_API_NEARBY_STOP_RADIUS_METERS:2500}
delijn.api.max-nearby-stops=${DELIJN_API_MAX_NEARBY_STOPS:20}
delijn.api.max-departures=${DELIJN_API_MAX_DEPARTURES:8}
```

Which subscription to use

Portal product	Use here?	Reason
Open Data Free	Yes	Includes Core and Search API operations used by the app.
De Lijn GTFS Static	No	Static ZIP feed; not used by this JSON RestClient flow.
De Lijn GTFS Realtime	No	Protobuf GTFS-RT feed; not parsed by this implementation.
Open Data Timetables Other formats	No	NeTEx/BLTAC formats are not parsed here.

Setup notes

- Set DELIJN_API_KEY in a local .env file or deployment environment.
- Do not commit the API key to Git.
- Docker Compose already passes the main De Lijn variables to the backend containers.
- Store DELIJN_API_KEY as a protected/masked GitLab CI/CD variable when deployment needs live API data.
- Without a key, the route page still loads and explains that De Lijn is not configured.

Route Option Strategy

The main method is `DeLijnService.getRouteOptions(...)`.

Path 1 - Route planner endpoint

If `delijn.api.endpoints.route-options` is set, the service calls the De Lijn route planner endpoint directly, parses route options, filters out options before the selected departure time, sorts by departure time, and returns `DeLijnRouteOptionDto` values.

Path 2 - Fallback strategy

In the current project setup, `route-options` is empty by default, so this path is normally active.

1. Find candidate stops near the origin coordinates.
2. Search stops using the origin label text.
3. Find candidate stops near the destination coordinates.
4. Search stops using the destination label text.
5. Pick the first destination-side stop candidate.
6. For future dates, fetch scheduled departures.
7. For today's date, fall back to real-time departures if scheduled departures are empty.
8. Build route option cards from those departures.

Accuracy note

This fallback is useful for showing public-transport options, but it does not guarantee full A-to-B route planning.

Result balancing

`TravelGroupRouteSuggestionService` balances bus and tram options so one transport type does not hide the other. It picks the earliest bus and tram options when present, fills remaining slots by departure time, then sorts the final list again.

7

Frontend Behavior

Main script: travelgroup-route-suggestions.js.

On page load

- Reads route defaults from data-* attributes
- Initialises the form with those defaults
- Calls the backend API and shows skeleton loading cards while waiting
- Initialises the Leaflet map lazily after the first API response
- Draws origin and destination markers
- Draws a straight route line between the two points
- Renders a route summary text block and horizontal De Lijn departure cards

User actions

- Submit the form to search with a new origin, destination, or departure time
- Swap origin and destination
- Reset to group defaults
- Use now to set the current time
- Click a route card to highlight it on the map
- Open a card in Google Maps transit mode

Map behavior

The map uses Leaflet with OpenStreetMap tiles. It displays origin and destination markers, a straight route line, a loading overlay, and a re-center button.

Map limitation

The line is a straight line between selected points. It is not real De Lijn bus or tram geometry.

Map Links and Live Pickup Sharing

The latest routing work also improves travel coordination around route suggestions.

Google and Apple Maps links

- Activity pages and travel group lists show compact map links near departure or meeting data.
- Travel group detail shows map links for activity location, group meeting point, and member shared locations.
- `travelgroup-map.js` fills Google and Apple map URLs from `data-maps-query` attributes.

Shared member location endpoints

Endpoint	Purpose
POST <code>/travelgroups/{groupId}/location</code>	Save a manual pickup/shared location for the current member.
POST <code>/travelgroups/{groupId}/location/live</code>	AJAX endpoint for temporary browser geolocation updates while the detail page is open.
POST <code>/travelgroups/{groupId}/location/clear</code>	Remove the current member's shared location without leaving the group.

Live sharing behavior

Browser GPS updates are throttled: tiny movements under 10 meters are ignored unless 30 seconds have passed. Leaving the page stops browser tracking, while the last saved location remains visible.

Public Transport Friend Matching

FriendMatchingService uses De Lijn data only as an extra layer. Normal matching still handles same activity, transport mode, nearby departure location, similar departure time, available seats, and exclusions for full, joined, pending, owned, invisible, or past groups.

When De Lijn matching runs

- The candidate group uses PUBLIC_TRANSPORT.
- The current member preference or explicit filter is compatible with public transport.
- DeLijnService is configured with an API key.
- Member, group, and destination coordinates are available.
- The request has not passed the per-request De Lijn comparison limit.

Public-transport score signals

Signal	Score	Reason
Same De Lijn origin stop	+30	Same De Lijn stop
Shared bus/tram line	+25	Compatible bus/tram route
Route arrives before event or arrival target	+15	Arrives before event starts
Route departure times within 30 minutes	+15	Similar departure time

Caching and rate-limit protection

- MatchingContext caches route-option lookups during one suggestion request.
- At most six De Lijn route comparisons are reserved per suggestion request.
- Runtime errors from De Lijn become empty route lists so normal matching can continue.

Helper Endpoints and Troubleshooting

Endpoint	Purpose
GET /api/delijn/nearby-stops	Find nearby stops if a nearby-stops endpoint path is configured.
GET /api/delijn/nearby-stops/debug	Debug the configured nearby-stops endpoint path.
GET /api/delijn/stop-suggestions	Frontend stop autocomplete.
GET /api/delijn/stops/{entityNumber}/{stopNumber}/scheduled-departures	Check scheduled departures for a stop/date.
GET /api/delijn/stops/{entityNumber}/{stopNumber}	Check details for one known stop.
GET /api/delijn/stops/{entityNumber}/{stopNumber}/probe	Probe known Core endpoint variants.
GET /api/delijn/stops/{entityNumber}/{stopNumber}/departures	Check real-time departures.

Common symptoms

Symptom	Likely cause	What to check
configured=false	Missing API key	Set DELIJN_API_KEY and restart.
401	Wrong key or inactive product	Use the Open Data Free subscription key.
404	Wrong endpoint path	Use probe/debug output to compare configured paths.
No future options	No scheduled departures for selected stop/date/time	Try another date/time or verify scheduled-departures path.

Demo Data and Manual Testing

Data.sql contains route-friendly demo data so the feature can be tested locally without creating every record by hand.

Seed data highlights

- Members with PUBLIC_TRANSPORT preferences and default departure coordinates such as Antwerp Central Station and Berchem Station.
- A De Lijn route lab activity for public-transport matching demos.
- Open public-transport groups and a full public-transport group to test exclusion behavior.
- Past groups to verify that old activities are not suggested.
- Belgian locations that make sense for De Lijn demos.

Manual test flow

1. Start the app with DELIJN_API_KEY configured.
2. Open a PUBLIC_TRANSPORT travel group and click the De Lijn route action.
3. Check loading skeletons, then scheduled or live option cards when data exists.
4. Try route reset, swap, use-now, stop suggestions, and external Google Maps transit links.
5. Open Google/Apple map links from group list, activity detail, and travel group detail.
6. Join a group, save a pickup point, and try live location sharing from the detail page.
7. Call /api/travelgroups/suggestions with transportMode=PUBLIC_TRANSPORT and check De Lijn match reasons when enough API data is available.

Known Limitations and Sources

Limitation	Reason
No guaranteed full A-to-B route planning	The active fallback uses nearby stops and departure data, not a real journey planner.
Map does not show actual bus/tram geometry	Leaflet draws a straight line between selected route points.
Destination stop is approximate	The fallback picks the first viable destination-side stop candidate.
Future dates depend on scheduled departures	The real-time fallback only activates when the selected date is today.
Text input fallback uses Nominatim	OpenStreetMap geocoding is used when De Lijn stop search cannot resolve typed text.
Live location is page-scoped	Browser live tracking stops when the detail page closes.
Nearby stops endpoint is optional	The direct nearby-stops path is empty by default and must be confirmed from the portal.

External references

Reference	URL
De Lijn Open Data portal	https://data.delijn.be/
Open Data V1 Core API listing	https://transportdata.be/nl/dataset/de-lijn-open-data-api-core-operations
Open Data V1 Search API listing	https://transportdata.be/dataset/de-lijn-open-data-api-search-operations